

Project Report

On

“Asteroid Impact AR”

Submitted in partial fulfillment of requirements for the award of Degree of

MASTER OF COMPUTER APPLICATIONS

in

DEPARTMENT OF COMPUTER APPLICATION

Engineering College Bikaner

Session: 2025-26

Submitted by:

Name:- Gunjan Vishwakarma

RollNumber:- 24CEBMC606

Semester:- IV

Department:- MCA

Submitted to:

Dr. Rakesh Poonia (HOD of MCA)

Engineering College Bikaner (Raj)

Guided by:

Name: Dr. Ankur Goswami

MCA Department

Engineering College Bikaner(Raj)



Engineering College Bikaner,

Pugal Road, RIICO Karni Industrial Area, Bikaner,
Rajasthan-33004

web: www.ecb.ac.in, Telephone: +91-1512253404,



Bikaner Technical University,
UCET Campus

Pugal Road, RIICO Karni Industrial Area,
Bikaner, Rajasthan-33004

web: www.btu.ac.in

Title	Sections Included
Preliminary Pages	Acknowledgement, Abstract, List of Figures, List of Tables
Chapter 1: Introduction	AR Overview, AR Gaming, Existing Systems, Objectives, Scope, Requirements, Limitations
Chapter 2: Proposed System	System Overview, Features, Functional Requirements, Non-Functional Requirements, Modules, Workflow
Chapter 3: Analysis & Design	System Analysis, Architecture, Use Case Diagram, Workflow Design, DFD, Health System, UI Design, GameState Architecture
Chapter 4: Implementation	Development Environment, Scene Management, Player Controller, Enemy System, Health System, UI, Audio, AR Integration
Chapter 5: Testing & Validation	Testing Methods, Functional Tests, Performance Testing, Device Testing, Validation Results
Chapter 6: Results & Discussion	Gameplay Results, AR Results, UI Results, Enemy Behavior, Screenshots, Discussion
Chapter 7: Drawbacks & Limitations	Platform Limitations, Difficulty Limitations, Optimization Issues
Chapter 8: Future Enhancements	Rewards, Multiplayer, Optimization, Cross-platform Support, Enhanced AR
Chapter 9: Conclusion	Project Summary, Final Conclusion
References	Bibliography / Sources

Centre for Cloud & DevOps Training Society, Bikaner

(A Complete IT Solutions for Open-Source Training and Software Development)

Amar Singh Pura, Behind Rajasthan Patrika Press, Bikaner-334001. Contact: +91-7742163889

Website: <https://www.cdtsbikaner.in>, Email: info@cdtsbikaner.com, cdtsbikaner@gmail.com

F.No. /CDTS/ 2026/MCA -Internship/149

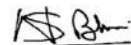
Date: 05-03-2026

TO WHOM IT MAY CONCERN

This is to certify that Mr. Gunjan Vishwakarma S/o Sh. Chander Shekhar Vishwakarma, student of MCA from Engineering College Bikaner, Bikaner (Rajasthan) has successfully completed an industrial training cum internship program on “**Android App Development**” and also worked on the same projects. The duration of the internship was 45 days between 19.01.2026 to 05-03-2026.

During the period of his training program with us found punctual, hardworking, and inquisitive.

We wish him success in life.



Narendra Singh Bhui

(Director)

CHAPTER 1: INTRODUCTION

1.1 Introduction to Augmented Reality

Augmented Reality (AR) is an interactive technology that integrates virtual objects into the real-world environment in real time. Unlike traditional gaming systems where users interact only with digital screens, AR combines computer-generated elements with physical surroundings to create immersive experiences. AR systems use cameras, sensors, and motion tracking algorithms to detect environmental conditions and accurately position virtual objects.

The advancement of mobile technology has significantly increased the adoption of AR in various domains including education, healthcare, manufacturing, retail, navigation, and entertainment. In recent years, AR-based games have gained popularity because they provide users with engaging and interactive experiences by blending virtual gameplay with real-world environments.

Modern smartphones equipped with advanced sensors and processing capabilities support AR frameworks such as ARCore and Unity AR Foundation, enabling developers to create interactive applications with environmental tracking and spatial awareness.

AR gaming introduces a new dimension of user engagement by allowing players to interact with virtual entities within physical spaces. This improves immersion and enhances gameplay realism compared to conventional 2D or 3D games.

1.2 Introduction to AR Gaming

Augmented Reality gaming is a category of digital entertainment where virtual game elements are superimposed onto the physical environment. AR games enable players to interact with virtual characters, objects, and scenarios while remaining aware of their real-world surroundings.

Traditional mobile games operate entirely within digital interfaces, whereas AR games convert surrounding environments into gameplay spaces. This capability increases user interaction and creates immersive experiences. AR gaming applications are widely used for entertainment, learning, training simulations, and educational purposes.

The increasing availability of AR-enabled smartphones has contributed to rapid growth in AR game development. Technologies such as motion tracking, environmental understanding, and object detection support realistic gameplay experiences.

1.3 Existing Systems and Need for Proposed System

Conventional space shooter games generally operate in static digital environments where users interact through fixed screens and predefined virtual worlds. Although such games provide entertainment, they often lack environmental immersion and real-world interaction.

Similarly, many AR applications emphasize object visualization rather than interactive gameplay involving dynamic enemy systems, survival mechanisms, and real-time user response.

The need for **Asteroid Impact AR** arises from combining immersive AR interaction with survival-based planetary defense gameplay. The system enables users to experience a space defense simulation within their surrounding environment where they must protect Earth against continuously spawning asteroids and UFO attacks.

The proposed system improves engagement by integrating:

- Real-time AR interaction
- Dynamic enemy spawning
- Continuous survival gameplay
- Projectile-based defense mechanisms
- Health and scoring systems
- Interactive user feedback

1.4 Problem Statement

Traditional mobile games provide limited environmental immersion because gameplay occurs entirely within digital interfaces. Existing AR applications often focus on visualization rather than interactive survival-based experiences.

Users require engaging AR experiences that combine real-world interaction with dynamic gameplay systems. Therefore, there is a need to develop an AR-based survival game where players actively defend Earth against incoming threats while interacting within their physical environment.

1.5 Objectives of the Project

The primary objectives of Asteroid Impact AR are:

1. To develop an Augmented Reality survival game using Unity and AR technologies.
2. To create an interactive gameplay environment where players defend Earth against asteroids and UFOs.
3. To implement real-time shooting mechanics and collision detection.
4. To develop health management and score tracking systems.
5. To provide an immersive AR experience on Android devices.
6. To create a scalable architecture that supports future enhancements and gameplay improvements.

1.6 Scope of the Project

The scope of Asteroid Impact AR includes the development of an Android-based AR survival game supporting:

- Real-time enemy spawning
- Projectile shooting mechanics
- Earth protection gameplay
- Health systems
- Scoring systems
- AR environment integration
- Endless survival gameplay

The current implementation does not include multiplayer support, cloud storage, progression systems, achievements, or reward mechanisms.

Future versions may introduce adaptive difficulty, player progression, rewards, and enhanced AR interactions.

1.7 Hardware and Software Requirements

Hardware Requirements

Componet	Requirement
Processor	Snapdragon/Equivalent

RAM	Minimum 4 GB
Camera	AR-supported mobile camera
Storage	Minimum 500 MB free
Device	Android smartphone

Software Requirements

Software	Version
Unity Engine	Unity 6
Language	C#
Platform	Android
IDE	Visual Studio
AR Framework	AR Foundation / ARCore
OS	Windows 10/11

1.8 Limitations of Current System

Current limitations include:

- No reward or achievement system
- No multiplayer support
- Static difficulty progression
- No persistent save system
- Performance optimization through object pooling is absent
- Android only implementation

CHAPTER 2: PROPOSED SYSTEM

2.1 Proposed System Overview

Asteroid Impact AR is an Android-based Augmented Reality (AR) survival game designed to provide an immersive planetary defense experience. The system places a virtual environment within the user's physical surroundings, where players must protect Earth from continuously spawning asteroids and hostile UFO attacks.

The gameplay follows an endless survival model in which threats continuously appear over time. The player interacts using touch-based shooting mechanics to destroy incoming enemies and prevent damage to Earth. Both the player and Earth possess health systems, and the game terminates when Earth health reaches zero.

The proposed system combines AR interaction with real-time gameplay mechanics including enemy spawning, collision detection, projectile shooting, score tracking, and health management.

The major objective of the system is to provide an engaging AR experience while demonstrating the implementation of interactive game systems using Unity and mobile AR technologies.

2.2 Features of the Proposed System

The developed system provides the following features:

1. Augmented Reality Environment

The game integrates virtual gameplay elements into the user's surrounding environment, creating immersive interaction.

2. Endless Survival Gameplay

Enemy entities continuously spawn, requiring the player to survive as long as possible.

3. Dynamic Enemy System

The game includes:

- Large asteroids
- Medium asteroids
- Small asteroids
- UFO enemies

Each enemy possesses different behaviors and attack mechanisms.

4. Shooting Mechanism

The player can fire projectiles toward enemies using touch input and raycasting.

5. Health Management

Separate health systems are maintained for:

- Player
- Planet Earth

Damage reduces health values, and health depletion leads to game-ending conditions.

6. Score Tracking

The score increases according to successful gameplay actions and survival duration.

7. Game Over System

When Earth health reaches zero, the game displays final score statistics and game over UI.

8. User Interface System

The interface displays:

- Score
- Health bars
- Crosshair
- Final score

- Navigation buttons
-

2.3 System Objectives

The proposed system aims to achieve:

- Immersive AR gameplay experience
 - Real-time interaction between player and virtual objects
 - Efficient enemy spawning mechanisms
 - Responsive shooting controls
 - Performance suitable for Android devices
 - Modular architecture supporting future expansion
-

2.4 Functional Requirements

Functional requirements describe operations the system must perform.

The system shall:

FR1 — Initialize AR Environment

The game must establish an AR environment during startup.

FR2 — Spawn Enemies

The system shall continuously generate asteroid and UFO enemies.

FR3 — Detect User Input

The system shall detect touch input and initiate shooting.

FR4 — Handle Collisions

The system shall calculate collisions between:

- Projectiles
- Asteroids
- UFOs
- Player
- Earth

FR5 — Manage Health

The system shall update health values dynamically.

FR6 — Update Score

The score must update according to gameplay events.

FR7 — Trigger Game Over

The game must terminate when Earth health reaches zero.

2.5 Non-Functional Requirements

Non-functional requirements define quality constraints.

Performance

The system should maintain acceptable frame rates on Android devices.

Reliability

Gameplay mechanics should operate without crashes during continuous execution.

Usability

Controls should remain intuitive for mobile users.

Scalability

Future gameplay systems should be incorporable without major architectural changes.

Compatibility

The application should function on AR-supported Android devices.

2.6 System Modules

The developed game contains multiple interconnected modules.

Module 1: AR Module

Responsible for:

- Environment detection
- Camera integration

- Virtual object placement
-

Module 2: Enemy Spawning Module

Responsible for:

- Large asteroid generation
 - Medium asteroid generation
 - Small asteroid generation
 - Spawn timing
-

Module 3: UFO Attack Module

Responsible for:

- UFO movement
 - Attack state transitions
 - Enemy shooting behavior
-

Module 4: Health System Module

Responsible for:

- Damage calculation
 - Health reduction
 - Health updates
 - Death conditions
-

Module 5: Score Management Module

Responsible for:

- Score tracking
 - UI updates
 - Final score generation
-

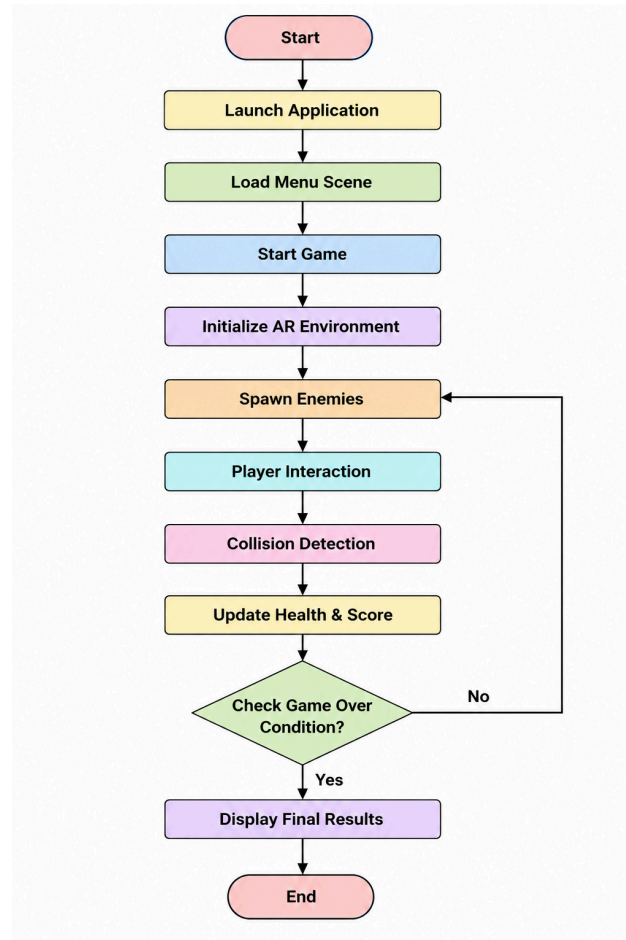
Module 6: User Interface Module

Responsible for displaying:

- Health bars
 - Score
 - Crosshair
 - Game over screen
-

2.7 System Workflow

The gameplay workflow of Asteroid Impact AR follows:



2.8 Advantages of Proposed System

The proposed system offers:

- Immersive AR interaction
 - Real-time gameplay
 - Endless survival mechanics
 - Simple mobile controls
 - Expandable architecture
 - Enhanced user engagement
-

2.9 Constraints of Proposed System

Current constraints include:

- Android-only support
 - No multiplayer functionality
 - No cloud synchronization
 - Limited progression systems
 - Static enemy spawn intervals
-

CHAPTER 3 :ANALYSIS & DESIGN

3.1 System Analysis

System analysis involves identifying user requirements, gameplay mechanics, system behavior, and interactions between various modules. The Asteroid Impact AR system is designed as an endless survival-based AR defense game where users interact with virtual enemies in real-time.

The system consists of several interconnected modules including:

- AR Environment Module
- Player Control Module
- Enemy Spawning Module
- Health Management Module
- Score Management Module
- User Interface Module
- Game State Management Module

All modules interact to maintain continuous gameplay.

3.2 User Requirements Analysis

The user requires:

Functional Requirements

Users should be able to:

- ✓ Start gameplay from menu screen
- ✓ Interact through touch input
- ✓ Shoot enemies

- ✓ Defend Earth
 - ✓ View score updates
 - ✓ Monitor health status
 - ✓ Restart game after game over
-

Non-functional Requirements

The system should provide:

- ✓ Responsive controls
 - ✓ Stable performance
 - ✓ Real-time updates
 - ✓ Mobile compatibility
 - ✓ Immersive AR interaction
-

3.3 Use Case Diagram

The primary actor is:

Player

Possible interactions:

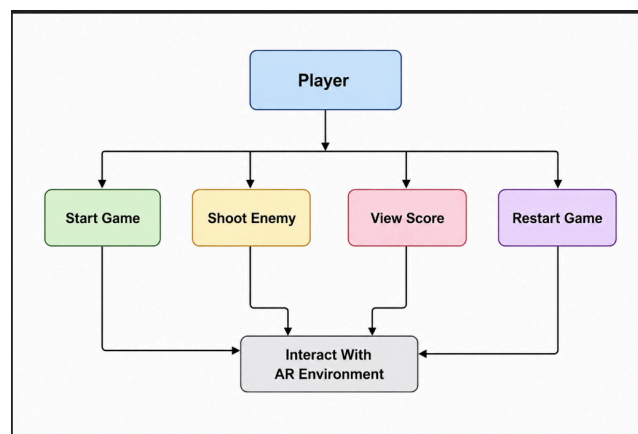


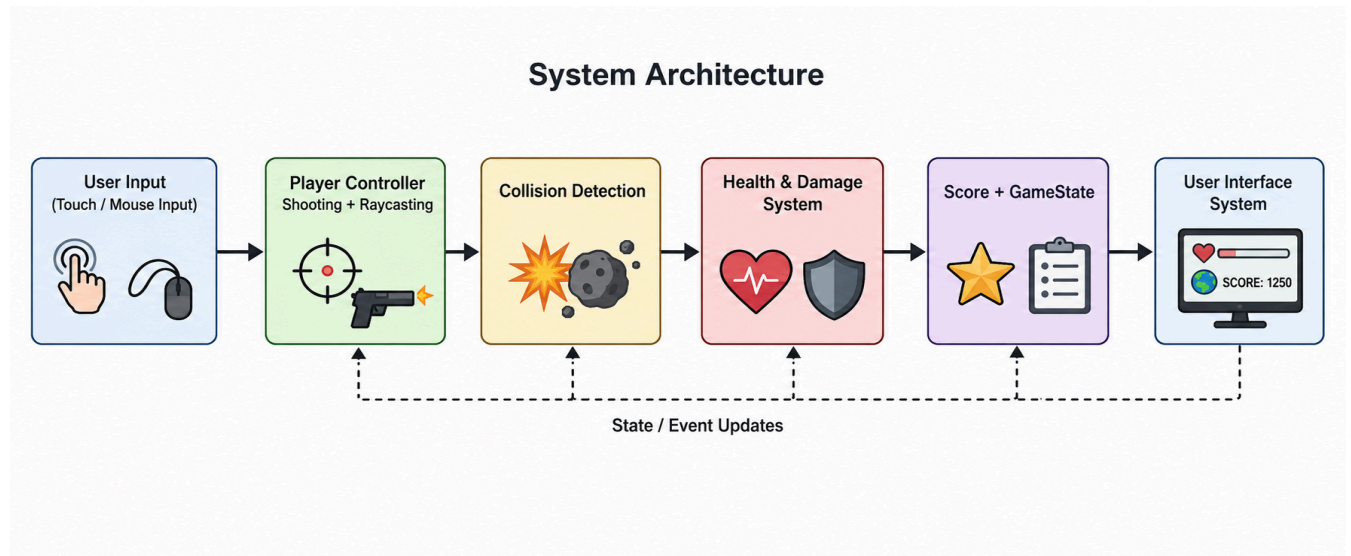
Figure 3.1: Use Case Diagram of Asteroid Impact AR

(Add proper UML diagram later.)

3.4 System Architecture

Asteroid Impact AR follows a modular architecture where different gameplay systems operate independently but communicate through shared game states and events.

System architecture:



3.5 Scene Architecture

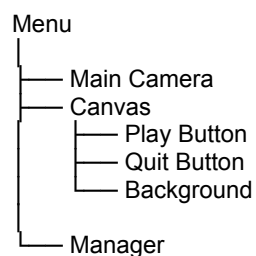
The project contains two primary scenes:

Menu Scene

Responsible for:

- Game initialization
- Scene loading
- Exit functionality

Structure:



Game Scene

Responsible for gameplay.

Structure:

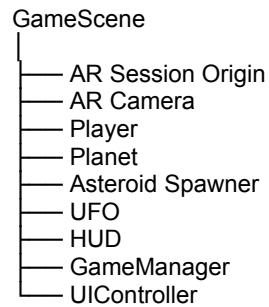
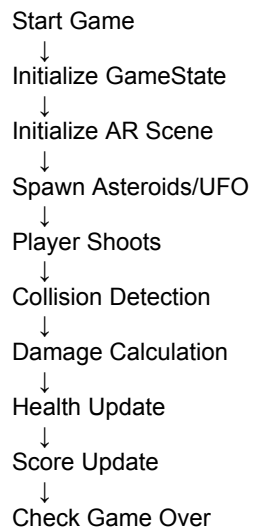


Figure 3.3: Game Scene Hierarchy

3.6 Gameplay Workflow Design

Gameplay follows:



This workflow continues until:

Earth Health = 0

Then:

Game Over
↓
Display Final Score
↓
Restart / Menu

3.7 Enemy Spawning Architecture

Enemy generation occurs continuously through the **SpawnColliderArea** system.

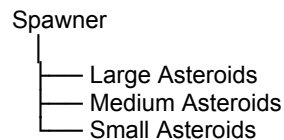
Current spawning:

Enemy Type	Spawn Delay
Large Asteroid	5 sec
Medium Asteroid	10 sec
Small Asteroid	8 sec

The spawning system:

1. Generates random positions inside collider bounds
2. Instantiates asteroid prefabs
3. Applies movement behavior
4. Sends enemies toward gameplay area

Architecture:



3.8 UFO Attack System Design

The UFO system uses **state-based behavior**.

States:

Idle
↓
Attacking

↓
Cooldown
↓
Idle

Behavior:

Idle State

- Waits random duration

Attack State

- Moves toward target trajectory
- Fires attacks

Cooldown State

- Resets attack cycle

This enables dynamic enemy behavior.

3.9 Health Management System

The health system manages:

1. Player Health
2. Earth Health

Initial values:

Max Health = 100

Damage processing:

Receive Damage
↓
Reduce Health
↓
Update UI
↓
Check Health ≤ 0
↓
Trigger Game Over

The Earth health reaching zero triggers:

GameState.GameOver = True

3.10 Damage System Design

The damage system performs:

Collision
↓
Compare Tags
↓
Identify Health Component
↓
Apply Damage
↓
Update Health

Supported interactions:

Source	Target
Asteroid	Player
Asteroid	Earth
UFO	Player
UFO	Earth

3.11 Score Management Design

Score is maintained through:

GameState
↓
IncreaseScore()
↓
Update UI

Responsibilities:

- Store score
 - Update display
 - Show final score
-

3.12 Game State Architecture

GameState functions as a centralized controller storing:

- Score
- Game speed
- Game over status
- Event triggers

Architecture:

GameManager



GameState



UI

Health

Spawner

Enemies

This enables communication between independent systems.

3.13 User Interface Design

The UI consists of:

Gameplay UI

Contains:

- Player health bar
 - Earth health bar
 - Score
 - Crosshair
-

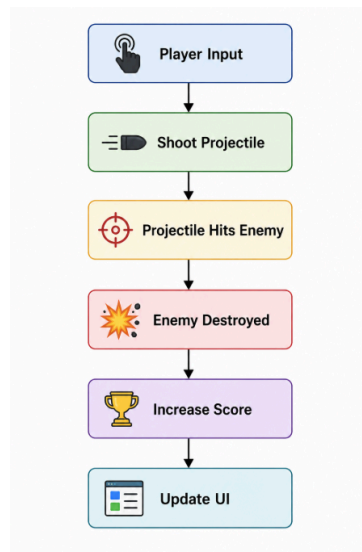
Game Over UI

Contains:

- Final score
 - Replay button
 - Home button
-

3.14 Sequence Diagram (Gameplay)

Example interaction:



3.15 Data Flow Diagram (Level 0)

Player
↓
Input System
↓
Gameplay Engine
↓
Health / Score System
↓
UI Output

(Add graphical DFD later.)

3.16 Design Limitations

Current architecture limitations:

- No object pooling
- Static spawn intervals
- No adaptive difficulty
- No persistence system
- Android-only deployment

CHAPTER 4 : IMPLEMENTATION

This chapter describes the practical implementation of Asteroid Impact AR using Unity, C#, and AR technologies. The system is developed using modular scripts where each component performs a dedicated functionality. The implementation focuses on gameplay interaction, enemy behavior, health systems, state management, and user interface integration.

4.1 Development Environment

The project was developed using the following tools and technologies:

Component	Technology Used
Game Engine	Unity 6
Programming Language	C#
Platform	Android
IDE	Visual Studio
UI Framework	Unity Canvas + TextMeshPro
AR Support	AR Foundation / ARCore (<i>confirm exact stack later</i>)
Version Control	Local Unity Project

The game is designed specifically for Android devices supporting AR capabilities.

4.2 Scene Management Implementation

The project contains two major scenes:

Scene 1: Menu

Purpose:

- Start gameplay
- Exit application

Implemented using:

MenuButtons.cs
SceneLoader.cs

Functions include:

PlayGame()
Menu()
QuitGame()

The menu system loads the gameplay scene and provides navigation control.

Scene 2: Game Scene

Purpose:

- Initialize AR environment
- Start gameplay systems
- Handle enemy spawning
- Update score and health

Contains:

GameManager
Player
Planet
Spawner
UFO
UI
GameState

4.3 Game Manager Implementation

The GameManager initializes gameplay variables.

Responsibilities:

- Reset score
- Reset GameOver status
- Initialize game speed
- Maintain TimeScale

Implementation flow:

```
Game Start
↓
Initialize GameState
↓
Reset Score
↓
Set Game Speed
↓
Start Gameplay
```

This ensures every session starts from a consistent state.

4.4 Game State Management

The project uses a **ScriptableObject-based GameState architecture**.

Stored information:

- Score
- Game speed
- Game over status
- Event notifications

The GameState acts as a centralized communication layer between modules.

Advantages:

- ✓ Reduced coupling
- ✓ Easier event handling
- ✓ Shared state access
- ✓ Modular implementation

Architecture:

GameState



Health
UI
Spawner
Enemies
Score

4.5 Player Controller Implementation

The player interacts through:

- Touch input
- Mouse input (Editor testing)

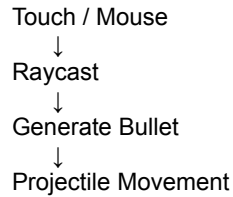
Implemented using:

Player.cs
PlayerMouseController.cs

Features:

Shooting System

Player input:



The system calculates shooting direction using raycasting and instantiates projectiles toward target positions.

Fire Rate Control

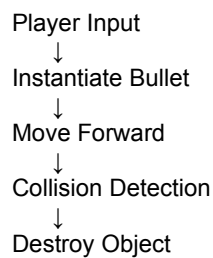
Cooldown implementation prevents unlimited firing.

Purpose:

- Improve gameplay balance
 - Control projectile generation
-

4.6 Projectile System Implementation

Projectile generation follows:



Associated scripts:

MoveForward.cs
CreateObject.cs
DestroyObject.cs

4.7 Enemy Spawning Implementation

Enemy generation is handled through:

[SpawnColliderArea.cs](#)

The spawning algorithm:

1. Generate random position
2. Instantiate enemy
3. Apply movement
4. Repeat after delay

Current enemy types:

Enemy	Spawn Interval
Large Asteroid	5 sec
Medium Asteroid	10 sec
Small Asteroid	8 sec

4.8 Asteroid Movement System

Asteroids move using:

MoveForward.cs

Movement:

Current Position
↓
Forward Direction
↓
Apply Speed
↓
Update Position

This creates continuous enemy movement toward gameplay space.

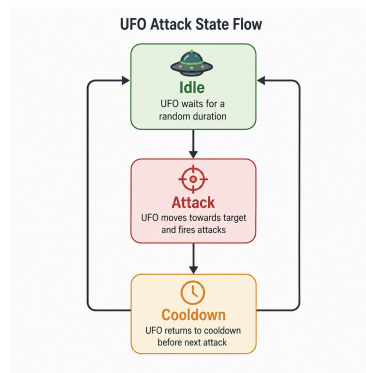
4.9 UFO Enemy Implementation

The UFO system uses state-based behavior.

Implemented using:

UFO.cs
UFOAttack.cs

States:



The UFO:

- Appears near player
- Moves through trajectories
- Shoots periodically
- Returns to cooldown state

This improves gameplay complexity.

4.10 Health System Implementation

Health management uses:

Health.cs
MakeDamage.cs
HealthItem.cs

Health supports:

Receive Damage

Collision
↓
Reduce Health
↓
Update UI

Gain Health

Pickup
↓
Increase Health
↓
Update UI

Supported entities:

Entity	Health
--------	--------

Player	Yes
--------	-----

Earth	Yes
-------	-----

4.11 Damage System

Damage occurs through collision detection.

Process:

Collision



Compare Tag



Identify Health Component



Apply Damage

Damage sources:

- Asteroids
- UFO attacks

Targets:

- Player
 - Earth
-

4.12 User Interface Implementation

The UI system displays:

Gameplay Screen

Contains:

- Score
- Crosshair
- Player health
- Earth health

Game Over Screen

Contains:

- Final score
- Replay button
- Menu button

Implemented using:

UIController.cs

Score Update Process

Increase Score



Update UI



Display New Score

4.13 Audio System Implementation

Audio management uses:

AudioSfx.cs
PlayAudioOnAwake.cs

Features:

- ✓ Random sound selection
- ✓ Volume control
- ✓ Pitch control
- ✓ Spatial audio
- ✓ Loop support

The implementation uses ScriptableObjects to maintain reusable audio configurations.

4.14 Object Lifecycle Management

Objects are automatically removed through:

DestroyAfterTime.cs
DestroyObject.cs
DestroyOnGameOver.cs

Purpose:

- Reduce unnecessary objects
- Maintain gameplay performance

4.15 Visual Effects Implementation

Implemented effects:

Rotation Effects

Rotate.cs

Used for:

- Asteroids
- Dynamic objects

Flash Effects

FlashEffect.cs

Purpose:

- Visual damage feedback
- Object response effects

4.16 AR Environment Integration

The gameplay operates within an AR environment.

Responsibilities:

- Camera tracking
- Environment detection
- Virtual object placement

The AR system allows gameplay objects to appear integrated within the user's surroundings.

4.17 Implementation Challenges

During development, challenges included:

1. Balancing enemy spawning
 2. Managing AR positioning
 3. Maintaining smooth gameplay
 4. Implementing responsive shooting
 5. Handling collisions accurately
-

4.18 Current System Limitations

Current implementation limitations:

- No object pooling
 - Static difficulty
 - No progression system
 - No persistent save data
 - Android-only support
-

This chapter should become around **8–10 pages** with screenshots of:

- Menu
 - Gameplay
 - Game Over
 - Hierarchy
 - Inspector panels
 - AR environment
-

CHAPTER 5 : TESTING & VALIDATION

Testing and validation are essential phases in software development to ensure that the developed system performs according to requirements. The Asteroid Impact AR application was tested to evaluate gameplay functionality, user interaction, health management, enemy spawning, scoring mechanisms, and overall stability on Android devices.

The objective of testing was to identify errors, verify expected behaviors, and improve application reliability.

5.1 Testing Objectives

The testing process aimed to verify:

- Correct scene transitions
 - Proper AR initialization
 - Accurate enemy spawning
 - Responsive player controls
 - Correct score updates
 - Proper health reduction
 - Stable gameplay execution
 - Correct game over conditions
-

5.2 Testing Methodology

The following testing methods were used:

Functional Testing

Verifies whether gameplay features perform as intended.

Integration Testing

Checks interaction between modules:

Example:

```
graph TD; A[Player Input] --> B[Projectile Generation]; B --> C[Collision Detection]; C --> D[Damage System]; D --> E[Health Update]; E --> F[UI Update];
```

Performance Testing

Evaluates:

- Responsiveness
- Gameplay smoothness
- Frame stability

Device Testing

Tests performed on Android devices supporting AR functionality.

5.3 Functional Test Cases

Test Case 1 : Game Start

Parameter	Result
Test Objective	Verify game starts successfully
Input	Press Play Button
Expected Output	Load gameplay scene
Actual Output	Gameplay loaded
Status	Pass

Test Case 2 : Enemy Spawning

Parameter	Result
Test Objective	Verify asteroid spawning
Input	Start gameplay
Expected Output	Continuous enemy generation
Actual Output	Enemies spawned successfully
Status	Pass

Test Case 3 : Shooting System

Parameter	Result
Test Objective	Verify projectile creation
Input	Screen touch
Expected Output	Bullet generated
Actual Output	Bullet created correctly
Status	Pass

Test Case 4 :Damage System

Parameter	Result
Test Objective	Verify damage calculation
Input	Enemy collision
Expected Output	Health reduction
Actual Output	Health reduced
Status	Pass

Test Case 5 :Health Bar Update

Parameter	Result
Test Objective	Verify UI health update
Input	Receive damage

Expected Output	Update health bar
Actual Output	Updated correctly
Status	Pass

Test Case 6 :Score Update

Parameter	Result
Test Objective	Verify score changes
Input	Gameplay actions
Expected Output	Increase score
Actual Output	Score updated
Status	Pass

Test Case 7 :Game Over Condition

Parameter	Result
Test Objective	Verify game over trigger
Input	Earth health reaches zero
Expected Output	Show Game Over screen
Actual Output	Displayed correctly
Status	Pass

Test Case 8 :Scene Navigation

Parameter	Result
Test Objective	Verify Home button
Input	Press Home
Expected Output	Return to Menu
Actual Output	Menu loaded
Status	Pass

5.4 Performance Testing

Performance testing was conducted to evaluate gameplay smoothness and responsiveness during continuous enemy generation.Observed performance criteria:

Parameter	Observation
Scene Loading	Acceptable
Input Response	Responsive
Enemy Generation	Stable
UI Updates	Immediate
Gameplay Continuity	Stable

The application maintained acceptable responsiveness during testing on Android devices.

5.5 AR Environment Testing

Testing objectives:

Verify:

- ✓ AR initialization
- ✓ Virtual object positioning
- ✓ Camera tracking
- ✓ Environment interaction

Observed results:

The game successfully positioned virtual elements relative to the surrounding environment and maintained gameplay interaction during movement.

5.6 Device Testing

Testing environment:

Parameter	Details
Platform	Android
Build Type	APK

Testing Method	Real Device
Result	Successful

5.7 Error Analysis

Common issues encountered during development included:

Issue 1 :Enemy Collision Behavior

Problem:

Unexpected collisions affecting gameplay.

Solution:

Adjusted collider settings and collision handling logic.

Issue 2 :AR Positioning Stability

Problem:

Virtual object placement required adjustment.

Solution:

Improved positioning behavior relative to environment.

Issue 3 :Shooting Accuracy

Problem:

Projectile direction inconsistency.

Solution:

Implemented raycasting-based shooting.

5.8 Validation Results

The developed system successfully satisfies primary requirements:

Requirement	Result
AR Gameplay	Achieved
Enemy Spawning	Achieved
Shooting System	Achieved
Health System	Achieved
Score Tracking	Achieved
Game Over System	Achieved
UI Updates	Achieved

5.9 Testing Limitations

Current testing limitations include:

- Limited device testing
- No large-scale compatibility testing
- No stress testing
- No long-duration performance analysis

Future testing may include broader hardware validation.

5.10 Summary of Testing

The testing process indicates that Asteroid Impact AR performs according to expected functionality and provides stable gameplay under normal operating conditions. Core systems including enemy spawning, health management, scoring, and UI updates operate correctly.

Good. I'll continue with all remaining chapters so you have a near-complete first draft structure. Later we can refine, add screenshots, diagrams, formatting, certificates, and references.

CHAPTER 6 : RESULTS & DISCUSSION

This chapter presents the outcomes obtained after implementing and testing the Asteroid Impact AR system. The results evaluate gameplay functionality, AR interaction, enemy behavior, user interface responsiveness, and overall user experience.

6.1 Gameplay Results

The developed application successfully implements an AR survival-based defense system where players protect Earth from incoming asteroids and UFO attacks.

The gameplay demonstrated:

- ✓ Continuous enemy spawning
- ✓ Responsive shooting mechanics
- ✓ Dynamic score updates
- ✓ Health reduction system
- ✓ Game over functionality
- ✓ AR environment integration

6.2 AR Environment Results

The application successfully integrates virtual gameplay objects into the surrounding environment through mobile AR capabilities.

Observed outcomes:

- Virtual objects maintained positional consistency
- Gameplay remained interactive during camera movement
- Earth and enemies appeared integrated within surroundings

The AR implementation improved immersion compared to conventional mobile gameplay.

6.3 User Interface Results

The interface displayed:

- Current score
- Earth health
- Player health
- Crosshair
- Final score
- Game over controls

The interface remained responsive throughout gameplay.

6.4 Enemy Behavior Results

Enemy systems demonstrated:

Asteroids

- Random spawning

- Continuous movement
- Collision response

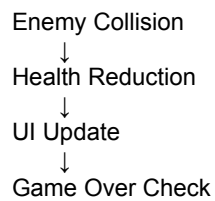
UFO

- State transitions
- Attack behavior
- Cooldown cycles

This increased gameplay complexity.

6.5 Health and Damage Results

The damage system correctly handled:



The Earth health reaching zero consistently triggered game termination.

6.6 Score System Results

The score system successfully:

- Stored score values
- Updated score display
- Presented final score

This provided user feedback and gameplay progression measurement.

6.8 Overall Results

The developed system achieved:

Objective	Result
AR Gameplay	Successful
Enemy Systems	Successful
Health Management	Successful
UI Integration	Successful
Endless Survival	Successful

6.9 Discussion

The developed game demonstrates the potential of combining AR technologies with survival gameplay mechanics. The implementation successfully creates an interactive environment while maintaining modular architecture suitable for future expansion.

The current system emphasizes immersion and interaction rather than progression systems or multiplayer functionality.

CHAPTER 7 : DRAWBACKS & LIMITATIONS

Despite successful implementation, several limitations exist within the current version.

7.1 Limited Platform Support

The application currently supports only Android devices compatible with AR functionality.

7.2 No Persistent Data Storage

The system does not store:

- High scores
- User progression
- Session history

All data resets after gameplay.

7.3 Absence of Reward System

The current implementation does not provide:

- Rewards
 - Achievements
 - Unlockables
-

7.4 Static Difficulty

Enemy spawning remains fixed rather than adaptive.

Example:

Enemy	Spawn Delay
-------	-------------

Large	5 sec
Medium	10 sec
Small	8 sec

Difficulty does not increase with survival duration.

7.5 No Object Pooling

Objects are repeatedly:

Instantiate()
Destroy()

This may affect performance during long gameplay sessions.

7.6 Limited Testing

Testing was performed on limited Android devices.

Broader compatibility testing remains necessary.

CHAPTER 8: FUTURE ENHANCEMENTS

The project architecture allows multiple improvements in future versions.

8.1 Adaptive Difficulty System

Enemy spawning could dynamically increase according to player survival time.

Benefits:

- Improved challenge
 - Increased engagement
-

8.2 Reward and Achievement System

Future versions may include:

- Coins
- Achievements
- Unlockables

- Progression systems
-

8.3 Multiplayer Support

Possible implementation:

- Cooperative mode
 - Competitive mode
-

8.4 Cloud Save System

Future systems may support:

- Saved progress
 - High scores
 - User profiles
-

8.5 Improved Optimization

Optimization techniques:

- Object pooling
 - Reduced memory allocation
 - Improved performance
-

8.6 Expanded Enemy Types

Additional enemies:

- Boss battles
 - Advanced UFO behavior
 - New asteroid mechanics
-

8.7 Cross-Platform Support

Potential support:

- iOS
 - Tablets
 - AR glasses
-

8.8 Enhanced AR Interaction

Possible improvements:

- Better environmental understanding
- Plane detection improvements

- More realistic placement
-

CHAPTER 9 : CONCLUSION

Asteroid Impact AR is an Augmented Reality survival game developed to provide an immersive planetary defense experience through real-time interaction between users and virtual gameplay elements. The project combines AR technologies with gameplay systems including enemy spawning, shooting mechanics, health management, scoring systems, and user interface integration.

The developed application successfully demonstrates how AR can transform traditional gaming experiences by integrating virtual environments with physical surroundings. The implemented modular architecture supports scalability and future enhancements.

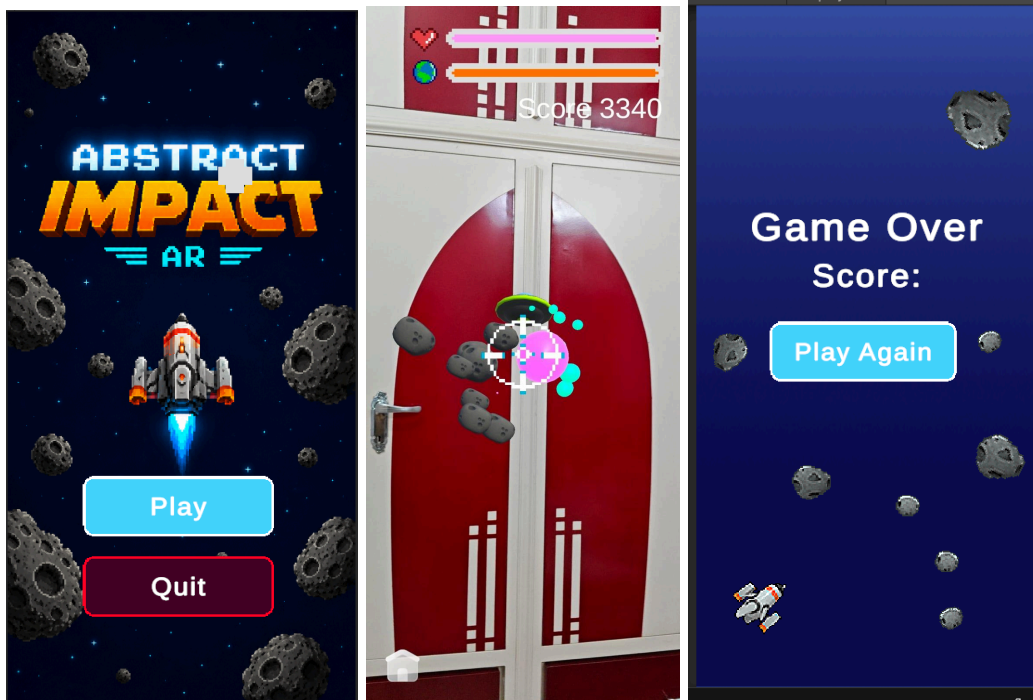
Testing results indicate that core systems operate correctly under normal conditions, providing stable gameplay and responsive interaction on Android devices. The project fulfills its primary objectives by delivering an engaging AR experience while establishing a foundation for future improvements including progression systems, adaptive difficulty, and multiplayer support.

Overall, Asteroid Impact AR illustrates the potential of combining Augmented Reality technologies with interactive survival gameplay and contributes toward understanding immersive mobile game development using Unity.

REFERENCES

1. Unity Technologies. *Unity Documentation*.
 2. Google Developers. *ARCore Documentation*.
 3. Unity Technologies. *AR Foundation Documentation*.
 4. Microsoft Documentation — C# Programming Language.
 5. TextMeshPro Documentation.
 6. Unity Learn Tutorials.
 7. Mobile AR development resources.
-

Some screenshots of the Game:



Scripts used in this application:

1. PlayerMouseController.cs:

```
CSharp
using System.Collections;
using UnityEngine;

public class PlayerMouseController : MonoBehaviour
{
    [Header("Bullet")]
    [SerializeField] private GameObject _objectToCreate;
    [SerializeField] private Transform _bulletSpawnPoint;
    [SerializeField] private float _fireRate = 0.3f;

    [Header("Crosshair")]
    [SerializeField] private RectTransform _crosshairUI;

    private Camera _mainCamera;
    private bool _canShoot = true;

    void Start()
    {
        _mainCamera = Camera.main;

        if (_mainCamera == null)
        {
            Debug.LogError("Main Camera not found!");
        }
    }
}
```

```

#if !UNITY_EDITOR
    // Keep crosshair centered on mobile
    if (_crosshairUI != null)
    {
        _crosshairUI.anchorMin = new Vector2(0.5f, 0.5f);
        _crosshairUI.anchorMax = new Vector2(0.5f, 0.5f);
        _crosshairUI.anchoredPosition = Vector2.zero;
    }
#endif

    void Update()
    {
#if UNITY_EDITOR
        UpdateEditorCrosshair();

        if (Input.GetMouseButtonDown(0))
        {
            TryShoot();
        }
#else
        // Mobile AR
        if (Input.touchCount > 0 && Input.GetTouch(0).phase == TouchPhase.Began)
        {
            TryShoot();
        }
#endif
    }

#if UNITY_EDITOR
    void UpdateEditorCrosshair()
    {
        if (_crosshairUI == null) return;
        _crosshairUI.position = Input.mousePosition;
    }
#endif

    void TryShoot()
    {
        if (!_canShoot) return;

        ShootBullet();
        StartCoroutine(ShootCooldown());
    }

    void ShootBullet()
    {
        if (_objectToCreate == null)
        {
            Debug.LogError("Bullet prefab missing!");
            return;
        }

        Ray ray;

#if UNITY_EDITOR
        // Shoot toward mouse

```

```

        ray = _mainCamera.ScreenPointToRay(Input.mousePosition);
#else
    // Shoot from screen center
    ray = _mainCamera.ViewportPointToRay(new Vector3(0.5f, 0.5f, 0));
#endif

    Vector3 targetPoint;

    if (Physics.Raycast(ray, out RaycastHit hit))
    {
        targetPoint = hit.point;
    }
    else
    {
        targetPoint = ray.GetPoint(50f);
    }

    Vector3 spawnPos = _bulletSpawnPoint != null ? _bulletSpawnPoint.position :
transform.position;
    Vector3 direction = (targetPoint - spawnPos).normalized;
    Quaternion rotation = Quaternion.LookRotation(direction);

    GameObject bullet = Instantiate(_objectToCreate, spawnPos, rotation);
    bullet.transform.forward = direction;
}

IEnumerator ShootCooldown()
{
    {
        _canShoot = false;
        yield return new WaitForSeconds(_fireRate);
        _canShoot = true;
    }
}
}

```

2. CreateObject.cs

```

CSharp
using UnityEngine;
using Random = UnityEngine.Random;

public class CreateObject : MonoBehaviour
{
    [SerializeField] private GameObject _objectToCreate;

    [SerializeField] private Transform _spawnPoint;

    [SerializeField] private bool _useSpawnPoint;

    [Range(0, 1)]
    [SerializeField] private float _chance = 1;

    [SerializeField] private Transform _referenceObjectRotation;

    [SerializeField] private int _amountOfObjects = 1;

    [SerializeField] private bool _randomizeInitialPositon;
}

```

```

[SerializeField] private float _xRandomizationFactor = 1;
[SerializeField] private float _yRandomizationFactor = 1;
[SerializeField] private float _zRandomizationFactor = 1;

public void CreateNewObject()
{
    for (int i = 0; i < _amountOfObjects; i++)
    {
        if (Random.value > _chance)
            continue;

        Quaternion rotation =
            _referenceObjectRotation == null
            ? Quaternion.identity
            : _referenceObjectRotation.rotation;

        Vector3 spawnPoint;

        if (_useSpawnPoint && _spawnPoint != null)
        {
            spawnPoint = _spawnPoint.position;
        }
        else
        {
            spawnPoint = transform.position;

            if (_randomizeInitialPositon)
            {
                spawnPoint.x += Random.Range(
                    -_xRandomizationFactor,
                    _xRandomizationFactor);

                spawnPoint.y += Random.Range(
                    -_yRandomizationFactor,
                    _yRandomizationFactor);

                spawnPoint.z += Random.Range(
                    -_zRandomizationFactor,
                    _zRandomizationFactor);
            }
        }

        GameObject clone =
            Instantiate(_objectToCreate, spawnPoint, rotation);

        clone.name = $"{clone.name} {clone.GetInstanceID()}";
    }
}

```

3. DestroyAfterTime.cs :

```

CSharp
using System.Collections;
using System.Collections.Generic;

```

```

using UnityEngine;

public class DestroyAfterTime : MonoBehaviour
{
    [SerializeField] private float _destroyDelay = 5;

    private void Start()
    {
        Destroy(gameObject, _destroyDelay);
    }
}

```

4. UIController.cs :

```

CSharp
using TMPro;
using UnityEngine;
using UnityEngine.UI;

public class UIController : MonoBehaviour
{
    [SerializeField] private TMP_Text _scoreText;
    [SerializeField] private GameState _gameState;
    [SerializeField] private Slider _playerHealth;
    [SerializeField] private Slider _planetHealth;
    [SerializeField] private GameObject _gameOverScreen;
    [SerializeField] private TMP_Text _finalScoreText;

    private void OnEnable()
    {
        _gameState.OnIncreaseScore.AddListener(UpdateScoreUI);
        _gameState.OnGameOver.AddListener(ShowGameOverScreen);
    }

    private void OnDisable()
    {
        _gameState.OnIncreaseScore.RemoveListener(UpdateScoreUI);
        _gameState.OnGameOver.RemoveListener(ShowGameOverScreen);
    }

    public void UpdateScoreUI(int newScore)
    {
        _scoreText.text = $"Score {newScore}";
    }

    public void UpdatePlayerHealthUI(int newHealth)
    {
        _playerHealth.value = newHealth;
    }

    public void UpdatePlanetHealthUI(int newHealth)
    {
        _planetHealth.value = newHealth;
    }

    public void ShowGameOverScreen()

```

```

    {
        _gameOverScreen.SetActive(true);
        _finalScoreText.text = $"Score: {_gameState.Score}";
    }
}

```

5. UFO.cs :

CSharp

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.Events;

public class UFO : MonoBehaviour
{
    public enum UFOStates
    {
        Idle,
        Attacking
    }

    [SerializeField] private UFOStates _currentState;

    [SerializeField]
    private List<Vector3> _trajectoryVectors =
        new List<Vector3>();

    [SerializeField] private int _trajectoriesPerSpawn = 2;

    [SerializeField] private float _spawnDistanceFromPlayer = 20;

    [SerializeField] private float _movementSpeed = 5;

    [SerializeField] private int _cooldownMinTime = 5;
    [SerializeField] private int _cooldownMaxTime = 15;

    [SerializeField] private GameState _gameState;

    [SerializeField] private AudioSource _ufoOnScene;

    [SerializeField] private UnityEvent OnDie;
    [SerializeField] private UnityEvent OnStartAttacking;
    [SerializeField] private UnityEvent OnStopAttacking;

    private Transform _player;

    private Coroutine _attackRoutine;

    public UFOStates CurrentState
    {
        get => _currentState;

        set
        {

```



```

        _currentState = value;

        if (_currentState == UFOStates.Attacking)
        {
            OnStartAttacking?.Invoke();
        }
        else
        {
            OnStopAttacking?.Invoke();
        }
    }
}

private void Start()
{
    GameObject playerObject =
        GameObject.Find("Player");

    if (playerObject != null)
    {
        _player = playerObject.transform;
    }

    CurrentState = UFOStates.Idle;
}

private IEnumerator IdleRoutine()
{
    transform.position = Vector3.one * 1000f;

    _trajectoryVectors.Clear();

    yield return new WaitForSeconds(
        Random.Range(
            _cooldownMinTime,
            _cooldownMaxTime));

    CurrentState = UFOStates.Attacking;
}

public void StartCooldown()
{
    StartCoroutine(IdleRoutine());

    _ufoOnScene.StopAudio();
}

public void StartAttacking()
{
    if (_player == null)
        return;

    transform.position = GetNewPositionVector();

    _trajectoryVectors.Clear();

    for (int i = 0; i < _trajectoriesPerSpawn; i++)
    {

```

```

        _trajectoryVectors.Add(
            GetNewPositionVector());
    }

    _ufoOnScene.PlayAudio(gameObject);

    if (_attackRoutine != null)
    {
        StopCoroutine(_attackRoutine);
    }

    _attackRoutine =
        StartCoroutine(AttackMovement());
}

private Vector3 GetNewPositionVector()
{
    if (_player == null)
        return transform.position;

    Vector3 forwardDirection =
        _player.forward;

    forwardDirection.y = 0;

    Vector3 spawnCenter =
        _player.position +
        forwardDirection.normalized *
        _spawnDistanceFromPlayer;

    Vector3 randomOffset =
        new Vector3(
            Random.Range(-5f, 5f),
            Random.Range(-2f, 2f),
            0);

    return spawnCenter + randomOffset;
}

IEnumerator AttackMovement()
{
    for (int i = 0; i < _trajectoryVectors.Count; i++)
    {
        while (
            Vector3.Distance(
                transform.position,
                _trajectoryVectors[i]) > 0.5f
            &&
            !_gameState.GameOver)
        {
            yield return null;

            transform.position =
                Vector3.MoveTowards(
                    transform.position,
                    _trajectoryVectors[i],
                    Time.deltaTime * _movementSpeed);
        }
    }
}

```

```

        CurrentState = UFOStates.Idle;
    }

    public void Die()
    {
        OnDie?.Invoke();

        StopAllCoroutines();

        StartCooldown();

        _ufoOnScene.StopAudio();
    }
}

```

6.DestroyObject.cs :

6. DestroyObject.cs :

CSharp

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.Events;

public class DestroyObject : MonoBehaviour
{
    [SerializeField] private float _destroyDelay = 0.1f;
    [SerializeField] private UnityEvent OnDestroy;

    public void DestroyWithDelay()
    {
        // Debug.Log($"{gameObject.name} destroyed");
        OnDestroy?.Invoke();
        Destroy(gameObject, _destroyDelay);
    }
}

```

7.SpawnColliderArea.cs

7. SpawnColliderArea.cs :

CSharp

```

using System.Collections;
using UnityEngine;

public class SpawnColliderArea : MonoBehaviour
{
    [SerializeField] private GameObject _prefab;
    [SerializeField] private Collider _collider;
    [SerializeField] private float _spawnDelayTime = 1;
    [SerializeField] private GameState _gameState;
    private Vector3 _spawnPosition;
}

```

```

void Start()
{
    // start spawn
    StartCoroutine(SpawnRoutine());
}

IEnumerator SpawnRoutine()
{
    // continue loop while GameOver is false
    while (!_gameState.GameOver)
    {
        // get new position for asteroid
        _spawnPosition = GetSpawnPosition();

        // create a new asteroid from prefab
        GameObject clone = Instantiate(_prefab, _spawnPosition, Quaternion.identity);

        // change new asteroid's game object name
        clone.name = $"{clone.name}{(clone.GetInstanceID())}";

        // wait before looping again
        yield return new WaitForSeconds(_spawnDelayTime);
    }
}

Vector3 GetSpawnPosition()
{
    // get a random position from the bounds of the AsteroidSpawner's Box Collider
    // use this to position our new spawned asteroid
    return new Vector3(Random.Range(_collider.bounds.min.x, _collider.bounds.max.x),
        Random.Range(_collider.bounds.min.y, _collider.bounds.max.y),
        Random.Range(_collider.bounds.min.z, _collider.bounds.max.z));
}
}

```

8. Player.cs

```

CSharp
using System.Collections;
using UnityEngine;
using UnityEngine.Events;

public class Player : MonoBehaviour
{
    [SerializeField] private UnityEvent OnShoot;
    [SerializeField] private float _fireRate = 0.3f;

    private bool _canShoot = true;

    void Update()
    {
        bool shoot = Input.GetMouseButtonDown(0);
    }
}

```

```

        if (shoot && _canShoot)
        {
            OnShoot?.Invoke();

            _canShoot = false;

            StartCoroutine(EnableShooting());
        }
    }

    IEnumerator EnableShooting()
    {
        yield return new WaitForSeconds(_fireRate);

        _canShoot = true;
    }
}

```

9. MoveForward.cs

```

CSharp
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class MoveForward : MonoBehaviour
{
    [SerializeField] private float _moveSpeed = 5;
    private Transform _myTransform;

    void Start()
    {
        // Store the reference of the GameObject's transform
        _myTransform = transform;
    }

    void Update()
    {
        // Increment the _myTransform position vector by _moveSpeed and Time.deltaTime
        _myTransform.position += _myTransform.forward * (Time.deltaTime * _moveSpeed);
    }
}

```

10.GameManager.cs

```

CSharp
using UnityEngine;

public class GameManager : MonoBehaviour
{
    [SerializeField] private GameState _gameState;

    private float _cachedSpeed;
}

```

```

private void Awake()
{
    _gameState.Score = 0;
    _gameState.GameOver = false;

    _cachedSpeed = _gameState.GameSpeed;

    Time.timeScale = _cachedSpeed;
}

private void Update()
{
    if (_cachedSpeed != _gameState.GameSpeed)
    {
        _cachedSpeed = _gameState.GameSpeed;

        Time.timeScale = _cachedSpeed;
    }
}
}

```

11. UFOAttack.cs

CSharp

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.Events;

public class UFOAttack : MonoBehaviour
{
    [SerializeField] private float _fireCooldownTime = 0.3f;
    [SerializeField] private UnityEvent OnShoot;

    // Initiates attack routine when the component is enabled
    private void OnEnable()
    {
        StartCoroutine(AttackRoutine());
    }

    IEnumerator AttackRoutine()
    {
        // Execute shooting loop while component remains active
        while (enabled)
        {
            // Invoke the shooting event
            OnShoot?.Invoke();

            // Pause execution based on cooldown duration
            yield return new WaitForSeconds(_fireCooldownTime);
        }
    }

    void Start()
    {
    }
}

```

```

    void Update()
    {
    }
}

```

12.HealthItem.cs

CSharp

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.Events;

public class HealthItem : MonoBehaviour
{
    [SerializeField] private string _tagToCompare = "Player";
    [SerializeField] private int _healingPower = 10;
    [SerializeField] private UnityEvent OnHeal;

    private void OnTriggerEnter(Collider other)
    {
        if (other.CompareTag(_tagToCompare))
        {
            if (other.TryGetComponent(out Health health))
            {
                health.GainHealth(_healingPower);
                OnHeal?.Invoke();
            }
        }
    }
}

```

13.MakeDamage.cs

CSharp

```

using UnityEngine;
using UnityEngine.Events;

// Inflicts damage upon a GameObject containing a Health component
public class MakeDamage : MonoBehaviour
{
    [SerializeField] private int _damagePower = 10;
    [SerializeField] private string _tagToCompare = "Player";
    [SerializeField] private UnityEvent OnTrigger;

    private void OnTriggerEnter(Collider other)
    {
        // Check if the collided object matches the required tag
        if (other.CompareTag(_tagToCompare))
        {
            // Attempt to retrieve the Health component from the target
            if (other.TryGetComponent(out Health health))
            {

```

```

        if (health.enabled)
        {
            // Apply damage and invoke associated events
            health.ReceiveDamage(_damagePower);
            OnTrigger?.Invoke();
        }
    }
}
}
}

```

14.SetInitialRotationTowardsTarget.cs

```

CSharp
using UnityEngine;

public class SetInitialRotationTowardsTarget : MonoBehaviour
{
    [SerializeField] private string _targetName;
    [SerializeField] private Vector3 _randomOffset;

    void Start()
    {
        // Locate the target within the scene hierarchy
        GameObject targetObject = GameObject.Find(_targetName);

        // Verify target existence before proceeding
        if (targetObject == null) return;

        // Calculate direction toward the target position
        Vector3 direction = targetObject.transform.position - transform.position;

        // Apply randomization based on the defined offset values
        direction += new Vector3(Random.Range(-_randomOffset.x, _randomOffset.x),
            Random.Range(-_randomOffset.y, _randomOffset.y),
            Random.Range(-_randomOffset.z, _randomOffset.z));

        // Establish the initial rotation toward the calculated direction
        transform.rotation = Quaternion.LookRotation(direction, Vector3.up);
    }
}

```

15. GameState.cs

```

CSharp
using UnityEngine;
using UnityEngine.Events;
using UnityEngine.SceneManagement;

[CreateAssetMenu(fileName = "GameData", menuName = "ScriptableObjects/CreateGameDataAsset")]
public class GameState : ScriptableObject
{
    public int Score;
    public UnityEvent<int> OnIncreaseScore;
    public UnityEvent OnGameOver;
}

```



```

public float GameSpeed = 1;
[SerializeField] private bool _gameOver;

public bool GameOver
{
    get => _gameOver;
    set
    {
        _gameOver = value;

        if (_gameOver)
            OnGameOver?.Invoke();
    }
}

public void IncreaseScore(int amount)
{
    Score += amount;

    OnIncreaseScore?.Invoke(Score);
}

public void LoadScene(string sceneName)
{
    SceneManager.LoadScene(sceneName);
}
}

```

16.PlayAudioOnAwake.cs

```

CSharp
using UnityEngine;

public class PlayAudioOnAwake : MonoBehaviour
{
    [SerializeField] private AudioSfx _audioSfx;

    private void Awake()
    {
        // Initiates audio playback upon object initialization
        _audioSfx.PlayAudio(gameObject);
    }
}

```

17.FlashEffect.cs

```

CSharp
using UnityEngine;

public class FlashEffect : MonoBehaviour
{
    [SerializeField] private Renderer[] _renderers;
    [SerializeField] private Color _flashColor;
    [SerializeField] private Color _originalColor;
}

```

```

[SerializeField] private float _cooldownSpeed = 5;
[SerializeField] private bool _playOnAwake;

private void Awake()
{
    if (_playOnAwake)
    {
        PlayFlash();
    }
}

public void PlayFlash()
{
    foreach (Renderer renderer in _renderers)
    {
        renderer.sharedMaterial.color = _flashColor;
    }
}

private void Update()
{
    if (_renderers.Length == 0)
        return;

    foreach (Renderer renderer in _renderers)
    {
        renderer.sharedMaterial.color =
            Color.Lerp(
                renderer.sharedMaterial.color,
                _originalColor,
                Time.deltaTime * _cooldownSpeed);
    }
}
}

```

18. DestroyOnGameOver.cs

```

CSharp
using UnityEngine;

public class DestroyOnGameOver : MonoBehaviour
{
    [SerializeField] private GameState _gameState;

    private void OnEnable()
    {
        // Subscribes to the game over event notification
        _gameState.OnGameOver.AddListener(DestroyG0);
    }

    private void OnDisable()
    {
        // Unsubscribes to prevent memory leaks and unexpected behavior
        _gameState.OnGameOver.RemoveListener(DestroyG0);
    }

    void DestroyG0()

```

```

    {
        // Automatically removes object when health conditions trigger game over
        Destroy(gameObject);
    }
}

```

19.Rotate.cs

```

CSharp
using UnityEngine;
using Random = UnityEngine.Random;

public class Rotate : MonoBehaviour
{
    [SerializeField] private Vector3 _speed;

    [SerializeField] private bool _randomizeInitialRotation;

    [SerializeField] private float _randomizationFactor = 2;

    private void Start()
    {
        // Apply initial randomization to rotation speed if enabled
        if (_randomizeInitialRotation)
        {
            float multiplier = Random.Range(0.5f, _randomizationFactor);
            _speed *= multiplier;
        }
    }

    void Update()
    {
        // Perform continuous rotation based on calculated speed and delta time
        transform.Rotate(_speed * Time.deltaTime, Space.Self);
    }
}

```

20. RandomizeUIElements.cs

```

CSharp
using UnityEngine;

public class RandomizeUIElements : MonoBehaviour
{
    [SerializeField] private RectTransform[] _elements;
    [SerializeField] private float _scaleMinValue = 0.1f;
    [SerializeField] private float _scaleMaxValue = 1f;
    [SerializeField] private float _positionThreshold = 5;

    void OnEnable()
    {
        // Iterates through the assigned UI elements to apply random transformations
        foreach (var e in _elements)
        {
            // ... (code for applying random transformations)
        }
    }
}

```

```

    {
        // Generates a random scale value within the defined range
        float localScale = Random.Range(_scaleMinValue, _scaleMaxValue);
        e.localScale = new Vector3(localScale, localScale, 1);

        // Calculates and applies a random position offset based on the threshold
        e.anchoredPosition = new Vector2(
            Random.Range(-_positionThreshold, _positionThreshold),
            Random.Range(-_positionThreshold, _positionThreshold));
    }
}

```

21. AudioSfx.cs

CSharp

```

using System;
using UnityEngine;
using Random = UnityEngine.Random;

[CreateAssetMenu(fileName = "Sfx", menuName = "ScriptableObjects/CreateNewSfxAsset")]
public class AudioSfx : ScriptableObject
{
    [Serializable]
    public struct AudioParametersStruct
    {
        public string AudioName;
        public AudioClip[] AudioClips;
        public float Volume;
        public float Pitch;
        public bool Loop;
        public float StartDelay;
        public float SpatialBlend;
    }

    public AudioParametersStruct AudioParameters;

    private AudioSource _audioSource;
    private GameObject _sourceGO;

    public void PlayAudio(GameObject parent)
    {
        if (AudioParameters.AudioClips.Length == 0) return;

        if (parent != null)
        {
            _audioSource = parent.GetComponent();
            if (_audioSource == null) _audioSource = parent.AddComponent();
        }
        else
        {
            _sourceGO = new GameObject($"Audio {AudioParameters.AudioName}");
            _audioSource = _sourceGO.AddComponent();
        }

        _audioSource.spatialBlend = AudioParameters.SpatialBlend;
    }
}

```

```

        _audioSource.clip = AudioParameters.AudioClips[Random.Range(0,
AudioParameters.AudioClips.Length)];
        _audioSource.volume = AudioParameters.Volume;
        _audioSource.pitch = AudioParameters.Pitch;
        _audioSource.loop = AudioParameters.Loop;

        _audioSource.PlayDelayed(AudioParameters.StartDelay);

        if (!_audioSource.loop && _sourceGO != null)
        {
            Destroy(_sourceGO, _audioSource.clip.length + 1f);
        }
    }

    public void PlayAudioWithoutParenting()
    {
        PlayAudio(null);
    }

    public void StopAudio()
    {
        if (_audioSource == null) return;
        if (_audioSource.isPlaying) _audioSource.Stop();
    }
}

```

22. SceneLoader.cs

```

CSharp
using UnityEngine.SceneManagement;

public class SceneLoader
{
    public static void LoadScene(string sceneName)
    {
        SceneManager.LoadScene(sceneName);
    }
}

```

23.MenuButtons.cs

```

CSharp
using UnityEngine;
using UnityEngine.SceneManagement;
using System.Collections;

public class MenuButtons : MonoBehaviour
{
    public void PlayGame()
    {
        // Initiates the primary gameplay session
        SceneManager.LoadScene("GameScene");
    }

    public void Menu()
    {
    }
}

```

```

        // Returns the user to the main menu interface
        SceneManager.LoadScene("Menu");
    }

    public void RestartScene()
    {
        // Triggers the scene reload sequence
        StartCoroutine(RestartRoutine());
    }

    IEnumerator RestartRoutine()
    {
        yield return null;

        // Identifies and reloads the currently active scene
        Scene currentScene = SceneManager.GetActiveScene();
        SceneManager.LoadScene(currentScene.name);
    }

    public void QuitGame()
    {
        // Terminates the application execution
        Application.Quit();
    }

    #if UNITY_EDITOR
        // Disables play mode during editor testing
        UnityEditor.EditorApplication.isPlaying = false;
    #endif
    }
}

```